



فهرست مسائل

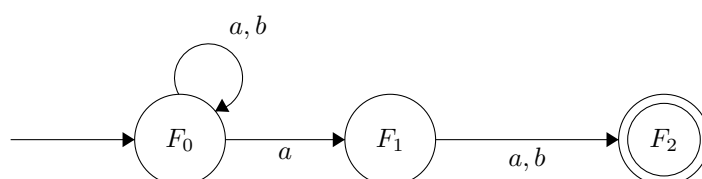
۱		مسئله ۱
۱		آ
۲		ب
۲		مسئله ۲
۳		آ
۳		ب
۵		مسئله ۳
۷		مسئله ۴
۷		آ
۷		ب
۸		ج
۹		مسئله ۵

پاسخ مسئله‌ی ۱.

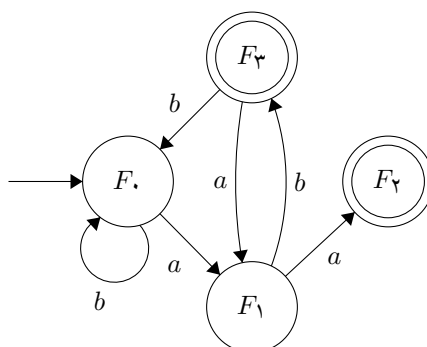
آ

$$(a + b)^* a (a + b)^*$$

ابتدا NFA را برای عبارت منظم داده‌شده رسم می‌کنیم:



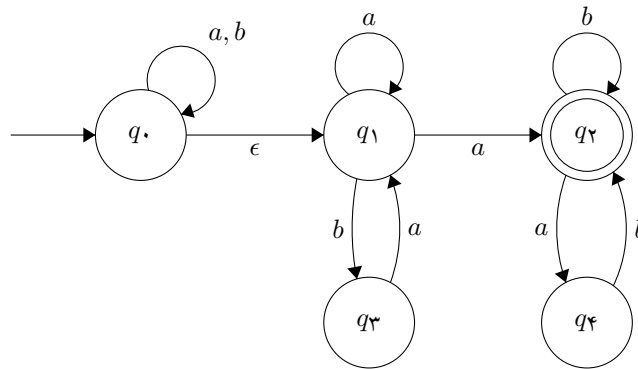
حال آن را تبدیل به DFA می‌کنیم. فرض می‌کنیم یک استتیت برای حالتی که آخرین حرف a است داریم. یک استتیت برای aa . یک استتیت هم برای ab . اگر آخر عبارت ما aa یا ab داشته باشیم، پس استتیت ما یک استتیت Final است.



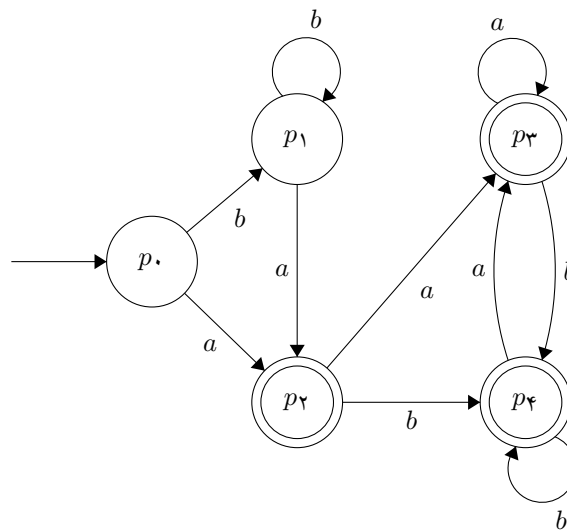
ب

$$(a + b)^*(a + ba)^*a(b + ab)^*$$

ابتدا NFA را برای عبارت منظم داده‌شده رسم می‌کنیم:



سپس با استفاده از الگوریتم تبدیل NFA به DFA، DFA آن را به دست می‌آوریم:



پاسخ مسئله‌ی ۲.

آ

این تساوی درست است. برای اثبات برابری دو عبارت منظم، نشان می‌دهیم که زبان‌های تولید شده توسط آن‌ها زیرمجموعه یکدیگر هستند.
جهت اول:

$$(R^*S^*)^* \subseteq (R+S)^*$$

می‌دانیم که $R \subseteq R+S$ و $S \subseteq R+S$. با اعمال عملگر استار که خاصیت یکنواختی دارد، خواهیم داشت:

$$R^* \subseteq (R+S)^* \quad \text{و} \quad S^* \subseteq (R+S)^*$$

بنابراین، الحاق این دو زبان نیز زیرمجموعه الحاق زبان دوم با خودش است:

$$R^*S^* \subseteq (R+S)^*(R+S)^* = (R+S)^*$$

در نهایت، مجدداً با اعمال عملگر استار روی هر دو طرف:

$$(R^*S^*)^* \subseteq ((R+S)^*)^* = (R+S)^*$$

جهت دوم:

$$(R+S)^* \subseteq (R^*S^*)^*$$

می‌دانیم که برای هر دو عبارت منظم R و S ، رابطه $R \subseteq R^*$ و $S \subseteq S^*$ برقرار است. در نتیجه، اجتماع آن‌ها نیز زیرمجموعه الحاق استارهای آن‌هاست:

$$R \cup S \subseteq R^*S^*$$

با اعمال عملگر استار روی هر دو طرف این رابطه:

$$(R+S)^* \subseteq (R^*S^*)^*$$

چون هر دو طرف زیرمجموعه یکدیگر هستند، تساوی برقرار است:

$$(R^*S^*)^* = (R+S)^*$$

ب

این تساوی نادرست است. با جایگذاری $R = a$ و $S = b$ (روی الفبایی که حداقل a و b را دارد) دو زبان را مقایسه می‌کنیم.

سمت چپ:

$$S(RS+S)^*R$$

با جایگذاری، عبارت برابر است با:

$$b(ab+b)^*a$$

عامل اول S برابر b است؛ پس هر کلمه‌ای در این زبان با b شروع می‌شود:

$$L(S(RS+S)^*R) \subseteq b\Sigma^*$$

برای نمونه با انتخاب توان صفر برای $(ab+b)^*$ داریم $ba = a$ که به زبان سمت چپ تعلق دارد.

سمت راست:

$$RR^*S(RR^*S)^*$$

با جایگذاری:

$$aa^*b(aa^*b)^*$$

زبان aa^*b همان a^+b است (حداقل یک a و سپس b); بنابراین

$$L(RR^*S(RR^*S)^*) = L(aa^*b) \cdot L(aa^*b)^* = (a^+b)^+$$

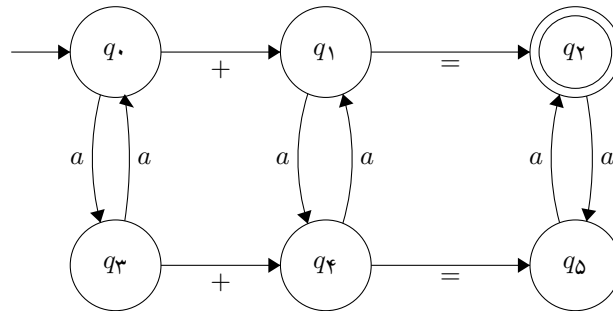
پس هر کلمه با a شروع می‌شود؛ مثلاً با یک بلوک aa^*b و توان صفر برای پرانتز آخر، رشته ab در این زبان است:

$$ab \in L(RR^*S(RR^*S)^*)$$

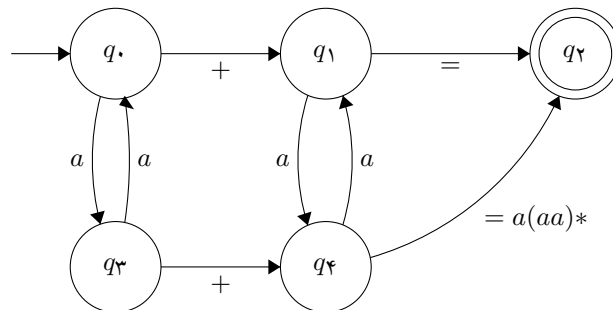
چون ab با a شروع می‌شود، $ab \notin b\Sigma^*$ و از شمول بالا $ab \notin L(S(RS + S)^*R)$. در حالی که ab در زبان سمت راست قرار دارد، پس دو زبان برابر نیستند و تساوی برقرار نیست.

پاسخ مسئله‌ی ۳.

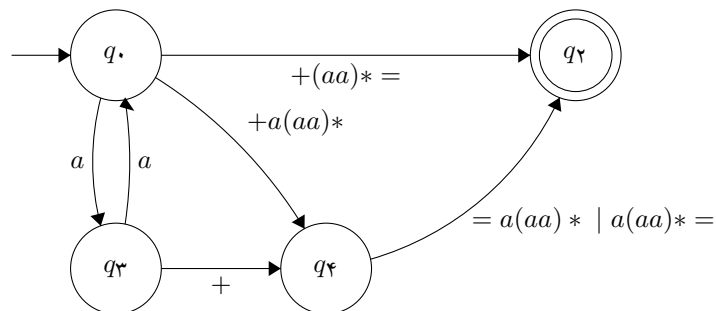
در این سوال قصد داریم تا NFA داده‌شده را به یک عبارت منظم تبدیل کنیم.



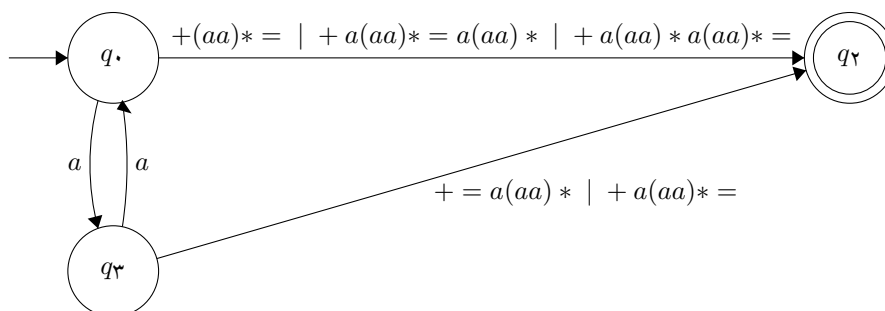
حال راس q_5 را حذف می‌کنیم:



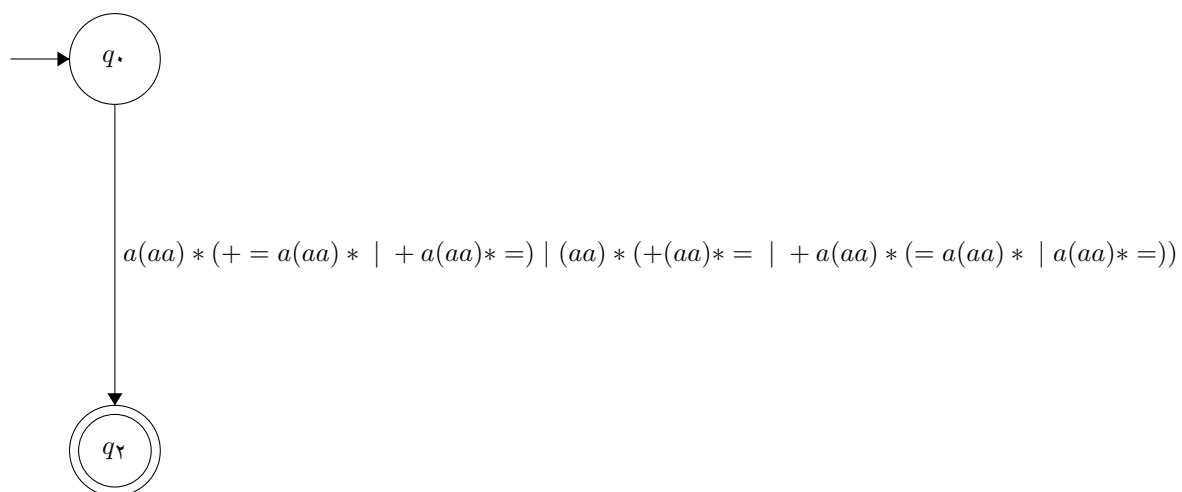
حال با حذف راس q_1 داریم:



حال با حذف راس q_4 داریم:



حال با حذف راس q_3 داریم:



پاسخ مسئله‌ی ۴.

آ

می‌خواهیم نشان دهیم

$$L_{\text{prefix}} = \{x \mid \exists y \in \Sigma^* xy \in L\}$$

با فرض منظم بودن L ، خودش نیز منظم است.

فرض کنیم $M = (Q, \Sigma, \delta, q_0, F)$ یک DFA برای L باشد. برای هر حالت $q \in Q$ تعریف می‌کنیم q به‌پذیرش‌رسیدنی است هرگاه مسیری از q به برخی حالت $f \in F$ وجود داشته باشد؛ یعنی برای بعضی رشته $z \in \Sigma^*$ داشته باشیم $\delta^*(q, z) \in F$.

$$F' = \{q \in Q \mid \text{به‌پذیرش‌رسیدنی است}\}$$

آنگاه $M' = (Q, \Sigma, \delta, q_0, F')$ دقیقاً L_{prefix} را می‌پذیرد: اگر $x \in L_{\text{prefix}}$ آنگاه برای بعضی y داریم $\delta^*(q_0, xy) \in F$ ؛ با قرار دادن $q = \delta^*(q_0, x)$ از q با خواندن y به حالت پذیرش می‌رسیم، پس $q \in F'$ و x توسط M' پذیرفته می‌شود. برعکس، اگر $\delta^*(q_0, x) \in F'$ آنگاه از $\delta^*(q_0, x)$ به بعضی $f \in F$ می‌توان رسید، یعنی $\exists y$ با $\delta^*(\delta^*(q_0, x), y) \in F$ و بنابراین $xy \in L$ پس L_{prefix} منظم است.

ب

زبان

$$L_{\downarrow} = \{x \mid \exists y, |x| = |y|, xy \in L\}$$

نیمه‌ی اول آن دسته از رشته‌های زوج‌طول L است که طول اولین تکه با طول دوم تکه برابر است. با فرض منظم بودن L ، این زبان هم منظم است.

فرض کنید $M = (Q, \Sigma, \delta, q_0, F)$ یک DFA برای L باشد. بعد از خواندن x روی M به حالت $q = \delta^*(q_0, x)$ می‌رسیم. رشته y با $|y| = |x|$ است و بعد از x خوانده می‌شود؛ پس باید از همان حالت q با دقیقاً $|x|$ گام به یک حالت پایانی برسیم:

$$xy \in L \iff \delta^*(q_0, xy) \in F \iff \exists y, |y| = |x|, \delta^*(q, y) \in F.$$

پس کافی است هم‌زمان با خواندن ورودی x ، مشخص کنیم کدام حالت‌های M با مسیری به طول دقیقاً برابر طول پیشوند خوانده‌شده به F می‌رسند؛ هرگاه حالت فعلی q در آن مجموعه باشد، برای همان طول یک y مناسب وجود دارد.

یک NFA با مجموعه‌ی حالت‌های $Q \times \mathcal{P}(Q)$ می‌سازیم؛ هر حالت را به صورت $[q, S]$ می‌نویسیم که در آن $q \in Q$ و $S \subseteq Q$ معنی $[q, S]$ بعد از خواندن پیشوندی به طول k از ورودی این است:

$$\bullet \quad q = \delta^*(q_0, x) \text{ برای همان پیشوند } x \text{ به طول } k$$

$$\bullet \quad S \text{ برابر مجموعه‌ی همه‌ی حالت‌های } p \in Q \text{ است که با مسیری به طول دقیقاً } k \text{ از } p \text{ به برخی حالت پایانی می‌توان رسید:}$$

$$p \in S \iff \exists w \in \Sigma^k : \delta^*(p, w) \in F.$$

حالت آغازین را $[q_0, F]$ می‌گیریم (برای $k = 0$ تنها رشته خالی به طول صفر از هر $f \in F$ به F می‌رسد).

انتقال با نماد $a \in \Sigma$: از حالت $[q, S]$ به حالت $[q', S']$ می‌رویم که

$$q' = \delta(q, a),$$

و S' شامل هر حالت $r \in Q$ است که با یک گام به بعضی عضو قبلی S برود:

$$S' = \{r \in Q \mid \exists b \in \Sigma, \exists p \in S : \delta(r, b) = p\}.$$

یعنی اگر قبلاً با k گام از S به $p \in F$ می‌شد، حالا با یک گام از r به p و سپس آن k گام، از r با $k+1$ گام به F می‌رسیم؛ درست هنگامی که ورودی یک نماد دیگر خوانده و طول پیشوند برابر $k+1$ شده است.

مجموعه پذیرش را تعریف می‌کنیم:

$$\mathcal{F} = \{[q, S] \mid q \in S\}.$$

اگر بعد از خواندن x به $[q, S]$ رسیده باشیم، $|x| = k$ و $q \in S$ یعنی خود q هم با مسیری به طول k به F می‌رسد؛ پس $\exists y$ با $|y| = k$ که $\delta^*(q, y) \in F$ و بنابراین $xy \in L$ برعکس، هر $x \in L_p$ این NFA را به حالتی در $\mathcal{F}$ می‌برد.

چون تعداد حالت‌ها متناهی است، L_p منظم است.

ج

زبان

$$L = \{a^n b^m a^k \mid n = m \vee m \neq k\}$$

روی $\Sigma = \{a, b\}$ را در نظر می‌گیریم.

می‌گوییم L منظم نیست. فرض خلف: L منظم باشد و طول پمپینگ را p بگیریم. رشته

$$w = a^p b^p a^p$$

در L است، چون $n = m = p$. طبق لم پمپینگ، تجزیه $w = xyz$ وجود دارد با $|xy| \leq p$ ، $|y| > 0$ ، و برای هر $i \geq 0$ داریم $xy^i z \in L$. چون $|xy| \leq p$ و اول w فقط a است، رشته y فقط از نماد a تشکیل شده و برای بعضی $t \geq 1$ داریم $y = a^t$. پس

$$xy^t z = a^{p+t} b^p a^p.$$

در این رشته طول بلوک اول $n' = p + t$ ، طول بلوک b برابر $m' = p$ ، و طول بلوک آخر $k' = p$. شرط $n' = m'$ برقرار نیست ($p + t \neq p$ به‌خاطر $t \geq 1$). شرط $m' \neq k'$ نیز برقرار نیست چون $m' = k' = p$. پس $xy^t z \notin L$ ، که با لم پمپینگ ناسازگار است.

بنابراین فرض باطل است و L منظم نیست.

پاسخ مسئله‌ی ۵.

زبان داده‌شده روی $\Sigma = \{0, 1\}$ این است:

$$L = \{w \in \Sigma^* \mid w \in \Sigma^* 101 \wedge |w|_0 \equiv 2 \pmod 3\}.$$

یعنی w باید هم‌زمان با الگوی 101 به پایان برسد و تعداد نمادهای 0 در w باقیمانده 2 بر پیمانه 3 داشته باشد. این زبان اشتراک دو زبان منظم است؛ پس دو DFA مجزا را ترکیب می‌کنیم و یک DFA یکتا به‌دست می‌آید.

ماشین نهایی با ساخت ضرب دو DFA (ردیابی پسوند 101 و شمارش باقی‌مانده) ساخته می‌شود. در جدول زیر هر حالت به‌صورت $[\sigma, j]$ نوشته شده است: σ بلندترین پسوند مسیر است که پیشوند 101 باشد ($\sigma \in \{\epsilon, 1, 10, 101\}$) و $j \in \{0, 1, 2\}$ باقیمانده تعداد نماد 0 پس از خواندن همان پیشوند است.

حالت فعلی	با 0 به	با 1 به
$[\epsilon, 0]$	$[\epsilon, 1]$	$[1, 0]$
$[\epsilon, 1]$	$[\epsilon, 2]$	$[1, 1]$
$[\epsilon, 2]$	$[\epsilon, 0]$	$[1, 2]$
$[1, 0]$	$[10, 1]$	$[1, 0]$
$[1, 1]$	$[10, 2]$	$[1, 1]$
$[1, 2]$	$[10, 0]$	$[1, 2]$
$[10, 0]$	$[\epsilon, 1]$	$[101, 0]$
$[10, 1]$	$[\epsilon, 2]$	$[10, 1]$
$[10, 2]$	$[\epsilon, 0]$	$[10, 2]$
$[101, 0]$	$[10, 1]$	$[101, 0]$
$[101, 1]$	$[10, 2]$	$[101, 1]$
$[101, 2]$	$[10, 0]$	$[101, 2]$

تنها حالت پذیرش $[101, 2]$ است.

